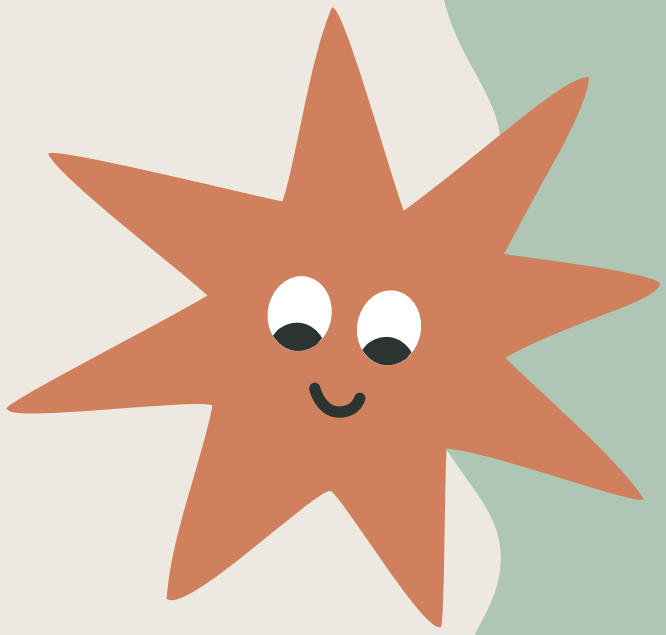
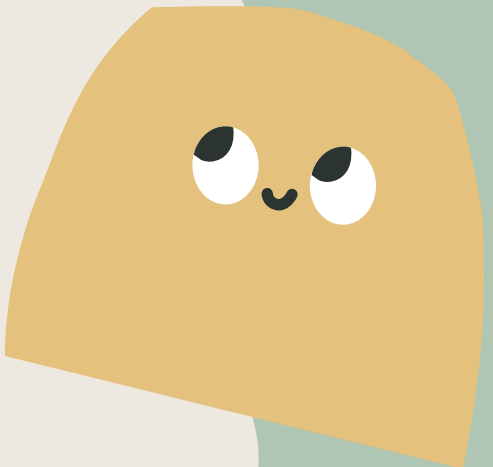
Implementing Breadth-First Search (BFS) in Python

Lecture 11





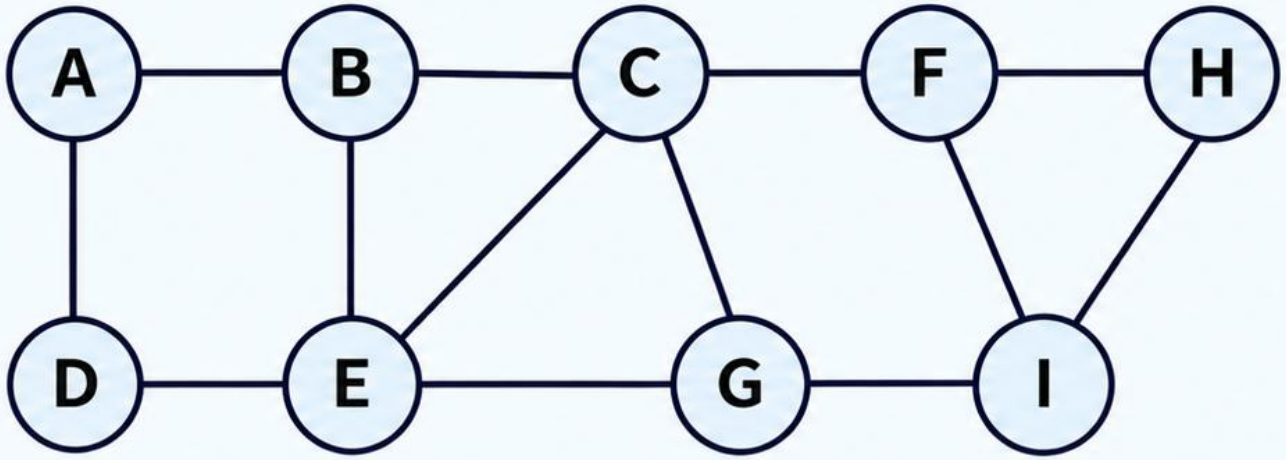
Learning Objectives

➢By the end of this lecture, students should be able to:

- Represent a graph using an adjacency list in Python.
- Implement BFS using a FIFO queue.
- Prevent duplicate visits using a visited set.
- Store parent relationships to reconstruct a solution path.
- Display the BFS traversal order and shortest path to a goal.
- Understand the basic time and space complexity of BFS.

Prepared By: Ehtisham Rasheed

Graph Representation



Python Representation (Adjacency List)

```python
# Graph represented as an adjacency list
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E'],
    'C': ['B', 'E', 'F', 'G'],
    'D': ['A', 'E'],
    'E': ['B', 'C', 'D', 'G'],
    'F': ['C', 'H', 'I'],
    'G': ['C', 'E', 'I'],
    'H': ['F', 'I'],
    'I': ['F', 'G']
}

# Display the graph
for node, neighbors in graph.items():
    print(f"{node}: {neighbors}")
```

Importing the Queue

- Python provides a deque data structure in the collections module. It allows efficient insertion and removal at either end
  - from collections import deque
- Create a queue:
  - queue = deque()
- Add a node to the rear:
  - queue.append('A')
- Remove a node from the front:
  - node = queue.popleft()
- This implements FIFO: **First In, First Out**.

Prepared By: Ehtisham Rasheed

Understanding the Required Variables in BFS

1 Queue

- Stores nodes that are waiting to be processed.
- Follows FIFO (First In, First Out).
- We add newly discovered neighbors at the end and remove from the front.

Example (after processing A):



Queue = [B, D, E]

2 Visited Set

- Stores nodes that have already been discovered (or added to the queue).
- Prevents the same node from being added multiple times.
- We mark a node as visited when we add it to the queue.

Example (after processing A):

visited = { A, B, D, E }

3 Parent Dictionary

- Stores the node from which each node was first discovered.
- Used later to reconstruct the solution path.
- The start node has no parent (*None*).

Example (after processing A):

Node	A	B	D	E
Parent	None	A	A	A

4 Traversal List

- Stores the order in which nodes are processed (removed from the queue).
- This gives us the BFS traversal order.

Example (after processing A):

traversal = [A]

Complete BFS Implementation in Python

Python Code

```python
from collections import deque

def bfs(graph, start, goal):
    # 1. Initialize data structures
    queue = deque([start])
    visited = {start}
    parent = {start: None}
    traversal = []

    # 2. Process the queue
    while queue:
        current = queue.popleft()
        traversal.append(current)

        # 3. Check if goal is found
        if current == goal:
            break

        # 4. Process all neighbors (in alphabetical order)
        for neighbor in sorted(graph[current]):
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)

    # 5. Reconstruct the solution path
    if goal not in parent:
        return traversal, None  # Goal not reachable
    path = []
    current = goal
    while current is not None:
        path.append(current)
        current = parent[current]
    path.reverse()
    return traversal, path
```

1 Initialize Data Structures

- queue: FIFO queue to store nodes to be processed.
- visited: set to keep track of discovered nodes.
- parent: dictionary to store the parent of each node.
- traversal: list to store the BFS traversal order.

2 Process the Queue

- Remove a node from the front of the queue.
- Add it to traversal order.

3 Check the Goal

- If the current node is the goal, we stop the search.

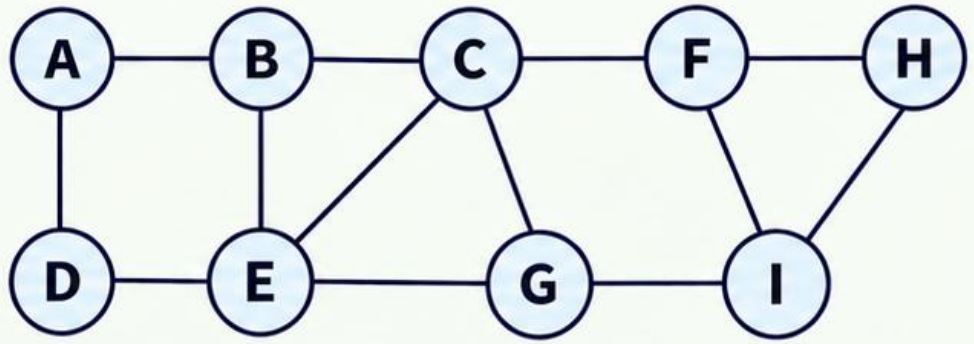
4 Process All Neighbors

- Consider neighbors in alphabetical order.
- If a neighbor is not visited:
  - mark it visited
  - set its parent
  - add it to the queue.

5 Reconstruct the Solution Path

- If goal is not in parent, return None.
- Otherwise, start from goal and follow parent links back to start.
- Reverse the path to get start → goal.

Example Graph



Run the BFS Function

```python
# Define the graph
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E'],
    'C': ['B', 'E', 'F', 'G'],
    'D': ['A', 'E'],
    'E': ['B', 'C', 'D', 'G'],
    'F': ['C', 'H', 'I'],
    'G': ['C', 'E', 'I'],
    'H': ['F', 'I'],
    'I': ['F', 'G']
}

# Run BFS from A to H
traversal, path = bfs(graph, 'A', 'H')

print("BFS Traversal Order:", traversal)
print("BFS Solution Path:", path)
```

Output

BFS Traversal Order

['A', 'B', 'D', 'C', 'E', 'F', 'G', 'H']

BFS Solution Path (from A to H):

['A', 'B', 'C', 'F', 'H']